BÀI TẬP LÝ THUYẾT
LẬP KẾ HOẠCH VÀ ĐỒ THỊ KẾ HOẠCH TRONG AI

Họ và tên	Võ Gia Kiệt
MSSV / Lớp	3123410182 / DCT1232

Bài 1. Khái niệm cơ bản trong GraphPlan

Phân tích định nghĩa và ý nghĩa chi tiết của các quan hệ mâu thuẫn (Mutex) cùng các phương pháp tìm kiếm trong thuật toán GraphPlan:

a) Interference (Xung đột do can thiệp):

• Định nghĩa: Hai hành động A1 và A2 ở cùng mức A_i được xem là có quan hệ Interference khi hiệu ứng xóa của hành động này làm mất đi một tiền điều kiện cần thiết của hành động còn lại.

• Ví dụ: A1 = Move(Left, Right) xóa mệnh đề CraneAt(Left), trong khi đây lại là tiền điều kiện để thực hiện A2 = PickUp(Left).

• Ý nghĩa: Quan hệ này ngăn hai hành động được thực hiện đồng thời khi một hành động làm mất điều kiện cần để hành động kia diễn ra.

b) Inconsistent Effects (Hiệu ứng không nhất quán):

• Định nghĩa: Hai hành động A1 và A2 tại cùng mức A_i có quan hệ Inconsistent Effects khi hiệu ứng của hành động này phủ định trực tiếp hiệu ứng của hành động còn lại (chẳng hạn A1 thêm literal P, còn A2 xóa P và tạo ra ¬P).

• Ví dụ: A1 = Gamble có hiệu ứng thêm ¬Rich (tức xóa Rich), trong khi A2 = No-op(Rich) lại duy trì mệnh đề Rich.

• Ý nghĩa: Quan hệ này bảo đảm mức trạng thái S_{i+1} không đồng thời chứa hai literal đối lập là P và ¬P.

c) Competing Needs (Tiền điều kiện cạnh tranh):

• Định nghĩa: Hai hành động A1 và A2 có quan hệ Competing Needs tại mức A_i khi tồn tại ít nhất một cặp tiền điều kiện tương ứng của chúng có quan hệ Mutex tại mức trạng thái liền trước S_i.

• Ví dụ: A1 = Eat(Cheese) yêu cầu Has(Cheese), còn A2 = Gamble yêu cầu Rich. Ở S_2, hai mệnh đề Has(Cheese) và Rich có quan hệ Mutex. Nguyên nhân là Has(Cheese) chỉ được tạo ra bởi Buy(Cheese), nhưng Buy(Cheese) lại xóa Rich. Vì vậy, Buy(Cheese) mutex với No-op(Rich), hành động có nhiệm vụ duy trì Rich. Do đó, Eat(Cheese) và Gamble có quan hệ Competing Needs tại A_2.

• Ý nghĩa: Quan hệ này bảo đảm các hành động chỉ được thực hiện đồng thời khi toàn bộ tiền điều kiện của chúng có thể cùng đúng.

d) Các phương pháp tìm kiếm kế hoạch trong Classical Planning:

1. Tìm kiếm trên không gian trạng thái (State-space search): Có thể tìm tiến từ trạng thái khởi đầu (Forward) hoặc tìm lùi từ trạng thái mục tiêu (Backward).
2. Tìm kiếm trên không gian kế hoạch (Plan-space search): Xây dựng kế hoạch theo thứ tự từng phần (Partial-Order Planning - POP).
3. Tìm kiếm dựa trên đồ thị kế hoạch (Planning Graph): Sử dụng thuật toán GraphPlan.

4. Mã hóa bài toán: Biểu diễn bài toán dưới dạng bài toán thỏa mãn Boolean (SATPlan) hoặc bài toán thỏa mãn ràng buộc (CSP).

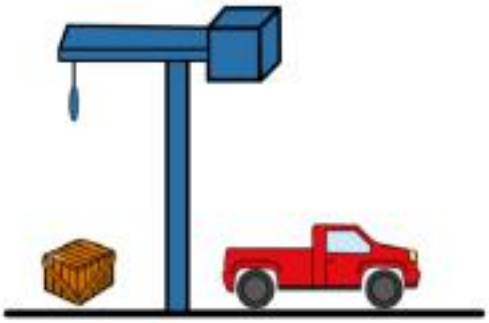
e) Giả định nới lỏng trong heuristic dựa trên GraphPlan:

• Giả định được nới lỏng: Trong đồ thị kế hoạch nới lỏng (Relaxed Planning Graph - RPG), ta bỏ qua các hiệu ứng xóa (Delete list / Delete effects) của hành động.

• Lợi ích so với tìm kiếm ngây thơ: Khi bỏ qua Delete list, một mệnh đề sau khi đạt được sẽ được xem là vẫn còn đúng ở các bước tiếp theo. Nhờ đó, việc ước lượng khoảng cách từ trạng thái hiện tại đến mục tiêu bằng heuristic được thực hiện nhanh hơn. Hàm h_max có tính admissible, còn h_FF (h_FAST-FORWARD) thường ước lượng chi phí thực tế hiệu quả nhưng không bảo đảm admissible. Các heuristic này giúp định hướng quá trình tìm kiếm và hạn chế việc mở rộng những nhánh kém triển vọng.

Bài 2. Lập kế hoạch ký hiệu - Bài toán cần cẩu

Tóm tắt bài toán: Ở trạng thái ban đầu, kiện hàng và cần cẩu đều ở vị trí Left, còn xe tải ở vị trí Right. Yêu cầu đặt ra là đưa kiện hàng lên xe tải tại Right, sau đó đưa cần cẩu trở về Left.



a) Khai báo mô hình bài toán (Objects, Predicates, States):

• Đối tượng (Objects) gồm: Crane (cần cẩu), Crate (kiện hàng), Truck (xe tải), Left (vị trí bên trái) và Right (vị trí bên phải).

• Các vị ngữ (Predicates) được sử dụng:

- CraneAt(loc): Biểu thị vị trí hiện tại của cần cẩu, trong đó loc ∈ {Left, Right}.
- CrateAt(loc): Cho biết kiện hàng đang nằm tại vị trí loc.
- EmptyCrane: Cho biết cần cẩu đang không giữ kiện hàng nào.
- Holding(crate): Biểu thị cần cẩu đang giữ/móc kiện hàng.
- Loaded(crate, truck): Cho biết kiện hàng đã được đưa lên xe tải.

• **Trạng thái ban đầu (S_0):** {CraneAt(Left), CrateAt(Left), EmptyCrane}

• **Trạng thái mục tiêu (G):** {Loaded(Crate, Truck), CraneAt(Left)}

b) Các toán tử hành động (Action Operators):

Action	Preconditions	Add List	Delete List
MoveCrane(from, to)	CraneAt(from)	CraneAt(to)	CraneAt(from)

PickUp(loc)	CraneAt(loc), CrateAt(loc), EmptyCrane	Holding(Crate)	EmptyCrane, CrateAt(loc)
LoadTruck()	CraneAt(Right), Holding(Crate)	Loaded(Crate, Truck), EmptyCrane	Holding(Crate)

c) Mô tả chi tiết các mức của Planning Graph (bao gồm các hành động duy trì No-op):

- S_0: Gồm các mệnh đề {CraneAt(Left), CrateAt(Left), EmptyCrane}.
- A_0: Bao gồm PickUp(Left), MoveCrane(Left, Right) và các hành động No-op tương ứng với 3 mệnh đề ở S_0.
- S_1: Gồm {CraneAt(Left), CrateAt(Left), EmptyCrane, Holding(Crate), CraneAt(Right)}.
- A_1: Gồm PickUp(Left), MoveCrane(Left, Right), MoveCrane(Right, Left) cùng các No-op ứng với 5 mệnh đề ở S_1. Chưa thể đưa LoadTruck vào mức này vì Holding(Crate) và CraneAt(Right) đang mutex tại S_1.
- S_2: Gồm {CraneAt(Left), CraneAt(Right), CrateAt(Left), EmptyCrane, Holding(Crate)}. Các mệnh đề vẫn giống S_1, nhưng quan hệ mutex đã giảm; cụ thể, Holding(Crate) và CraneAt(Right) không còn mutex.
- A_2: Bao gồm LoadTruck, MoveCrane(Left, Right), MoveCrane(Right, Left), PickUp(Left) và các No-op tương ứng với 5 mệnh đề tại S_2.
- S_3: Được tạo thành từ S_2 và bổ sung Loaded(Crate, Truck). Hai mệnh đề Loaded(Crate, Truck) và CraneAt(Left) vẫn mutex: LoadTruck và MoveCrane(Right, Left) cùng cần CraneAt(Right), đồng thời LoadTruck có Competing Needs với No-op(CraneAt(Left)).
- A_3: Gồm MoveCrane(Right, Left), No-op(Loaded(Crate, Truck)) và các hành động còn lại.
- S_4: Lúc này Loaded(Crate, Truck) và CraneAt(Left) không còn mutex, vì vậy tập mục tiêu G đã được thỏa mãn.
- *Kế hoạch thực hiện: PickUp(Left) → MoveCrane(Left, Right) → LoadTruck → MoveCrane(Right, Left).*

d) Phân tích các cặp hành động Mutex tiêu biểu tại mức A_0 và A_1:

1. Cặp Mutex ở mức A_0:

• **PickUp(Left) và MoveCrane(Left, Right):**

- **Loại Mutex:** Interference
- Giải thích: MoveCrane(Left, Right) xóa CraneAt(Left), trong khi CraneAt(Left) là điều kiện bắt buộc để thực hiện PickUp(Left).

• **PickUp(Left) và No-op(EmptyCrane):**

- **Loại Mutex:** Inconsistent Effects
- Giải thích: PickUp(Left) xóa EmptyCrane, còn No-op(EmptyCrane) có nhiệm vụ giữ nguyên mệnh đề EmptyCrane, nên hai hành động có hiệu ứng đối lập.

• **MoveCrane(Left, Right) và No-op(CraneAt(Left)):**

- **Loại Mutex:** Inconsistent Effects

- Giải thích: MoveCrane(Left, Right) loại bỏ CraneAt(Left), trái với hiệu ứng duy trì CraneAt(Left) của No-op.

• **PickUp(Left) và No-op(CrateAt(Left)):**

- **Loại Mutex:** Inconsistent Effects
- Giải thích: PickUp(Left) xóa CrateAt(Left), trong khi No-op(CrateAt(Left)) duy trì chính mệnh đề này.

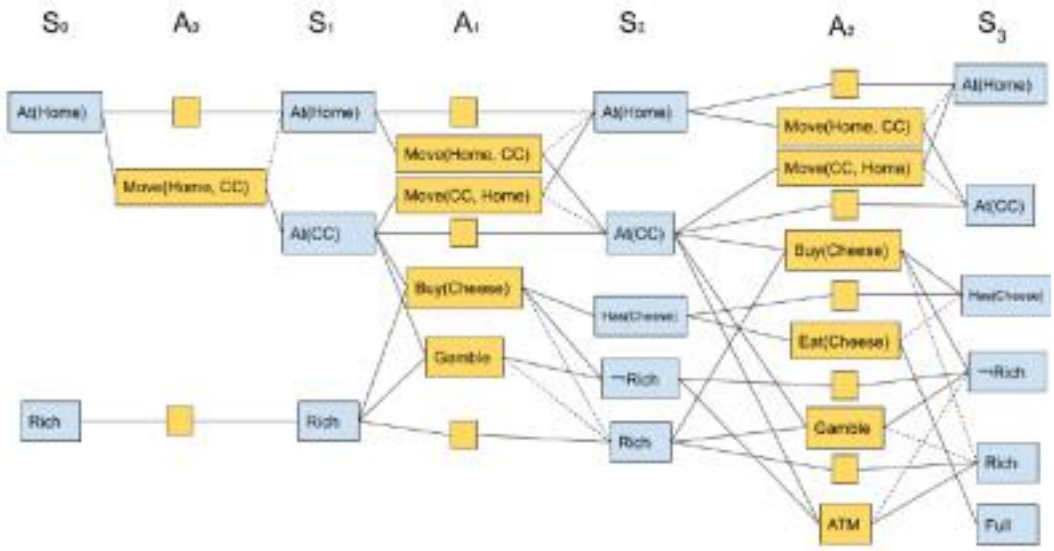
2. Cặp Mutex ở mức A_1:

- MoveCrane(Left, Right) và No-op(CraneAt(Left)): thuộc loại Inconsistent Effects, vì hành động MoveCrane xóa CraneAt(Left), còn No-op giữ nguyên mệnh đề đó.
- PickUp(Left) và MoveCrane(Left, Right): thuộc loại Interference do MoveCrane xóa CraneAt(Left), là tiền điều kiện của PickUp(Left).
- PickUp(Left) và MoveCrane(Right, Left): thuộc loại Competing Needs. PickUp yêu cầu CraneAt(Left), còn MoveCrane(Right, Left) yêu cầu CraneAt(Right); hai mệnh đề này mutex tại S_1.

Bài 3. Phân tích Mutex trong GraphPlan (Bài toán Pinky)

Hành động	Preconditions	Add List	Delete List
Move(A,B)	At(A)	At(B)	At(A)
Buy(Cheese)	At(ChuckyCheese), Rich	Has(Cheese), ¬Rich	
Gamble	At(ChuckyCheese), Rich	¬Rich	Rich
ATM	At(ChuckyCheese), ¬Rich	Rich	¬Rich
Eat(Cheese)	Has(Cheese)	Full	Has(Cheese)

Bài toán có trạng thái ban đầu S_0 = {Rich, At(Home)} và trạng thái mục tiêu G = {Full}.



Planning graph cho bài toán Pinky.

Mức	Hành động / Literal 1	Hành động / Literal 2	Loại Mutex	Giải thích chi tiết
A_0	Move(Home, CC)	No-op(At(Home))	Inconsistent Effects	Move(Home, CC) xóa At(Home), trong khi No-op(At(Home)) duy trì mệnh đề này. Vì vậy, hai hành động có hiệu ứng không nhất quán.
A_1	Buy(Cheese)	Gamble	Interface	Buy(Cheese) xóa Rich, là tiền điều kiện của Gamble. Hai hành động đều có thể được xét tại A_1 vì At(CC) và Rich cùng xuất hiện ở S_1 và không mutex.
A_2	Eat(Cheese)	Gamble	Competing Needs	Eat(Cheese) yêu cầu Has(Cheese), còn Gamble yêu cầu Rich. Tại S_2 , hai literal này mutex vì Has(Cheese) được tạo bởi Buy(Cheese), hành động đồng thời tạo ¬Rich, đối lập với Rich.

Bài 4. Classical Planning và GraphPlan - Valet Parking

Thành phần	ParkBehind(car1, car2)	ParkInNewRow(car)
Preconditions	ClearBehind(car1), ClearBehind(car2), ParkedBehind(car1, place)	ClearBehind(car), ParkedBehind(car, place), ¬ParkedBehind(car, curb)
Add List	ParkedBehind(car1, car2), ClearBehind(place), ¬ParkedBehind(car1, place), ¬ClearBehind(car2)	ParkedBehind(car, curb), ClearBehind(place), ¬ParkedBehind(car, place)
Delete List	ClearBehind(car2), ParkedBehind(car1, place)	ParkedBehind(car, place), ¬ParkedBehind(car, curb)

Dựa vào các toán tử hành động và planning graph mẫu của bài toán Valet Parking, ta có:

a) Xác định các hành động tương ứng tại ô 1-4 ở mức A_0:

• Ô 1: ParkInNewRow(B)

• Ô 2: N/A (không có hành động thỏa mãn các tiền điều kiện)

• Ô 3: ParkBehind(B, C)

• Ô 4: N/A (không có hành động thỏa mãn các tiền điều kiện)

b) Tập hợp các predicate ở mức S_1 nhận cạnh nối (add-edge) từ các hành động 1-4:

• Hành động 1 (ParkInNewRow(B)) tạo các predicate: {ParkedBehind(B, curb), ClearBehind(A), ¬ParkedBehind(B, A)}

• **Hành động 2:** N/A

• Hành động 3 (ParkBehind(B, C)) tạo các predicate: {ParkedBehind(B, C), ClearBehind(A), ¬ParkedBehind(B, A), ¬ClearBehind(C)}

• **Hành động 4:** N/A

c) Cặp hành động Mutex do Inconsistent Effects:

• Cặp hành động được chọn: Action 1 (ParkInNewRow(B)) và No-op(ParkedBehind(B, A)).

• Giải thích: ParkInNewRow(B) xóa ParkedBehind(B, A), đồng thời tạo ¬ParkedBehind(B, A). Điều này đối lập trực tiếp với No-op(ParkedBehind(B, A)), vốn duy trì ParkedBehind(B, A).

d) Cặp hành động Mutex do Interference:

• Cặp hành động được chọn: Action 1 = ParkInNewRow(B) và Action 3 = ParkBehind(B, C).

• **Loại Mutex:** Interface

• Giải thích: ParkBehind(B, C) xóa ParkedBehind(B, A), trong khi đây là tiền điều kiện của ParkInNewRow(B). Hai hành động không tạo ra các hiệu ứng đối nghịch trực tiếp, vì vậy đây là trường hợp Interference.

e) Điều kiện GraphPlan "leveled off":

Đáp án: C. Hai mức liên tiếp của planning graph có nội dung giống nhau, bao gồm tập literal, tập hành động và các quan hệ mutex.

Giải thích: Đồ thị được xem là đã "leveled off" khi việc mở rộng sang mức tiếp theo không làm xuất hiện thêm literal hoặc hành động mới, đồng thời các quan hệ mutex cũng không còn thay đổi. Nói cách khác, planning graph đã đạt trạng thái ổn định.

Bài 5. Xây dựng GraphPlan và trích xuất kế hoạch

Bài toán có trạng thái ban đầu S_0 = {A} và tập mục tiêu G = {C, D, E}.

	op1	op2	op3	op4	op5	op6
Precondition	A	B	A	A	A	A
Add	B	C, D, E	C	D	E	E, ¬A
Delete						

a) Quá trình phát triển Planning Graph mở rộng chuẩn (Parallel Planning Graph):

• Mức trạng thái S_0: Tập mệnh đề ban đầu là {A}.

• Mức hành động A_0: Các hành động có tiền điều kiện thỏa mãn tại S_0 gồm {op1, op3, op4, op5, op6}, cùng với No-op(A).

• Mức trạng thái S_1: Sau khi thực hiện các hành động ở A_0, tập mệnh đề thu được là {A, B, C, D, E, ¬A}. Như vậy, cả ba mục tiêu {C, D, E} đều đã xuất hiện tại S_1.

• Kiểm tra mutex giữa các mục tiêu tại S_1: Không có cặp literal nào trong G = {C, D, E} bị mutex. Chẳng hạn, C, D và E lần lượt có thể được tạo bởi op3, op4 và op5; các hành động này không mutex với nhau tại A_0.

• Kết luận: Có thể dừng việc mở rộng planning graph tại S_1 và chuyển sang bước trích xuất kế hoạch, không cần xây dựng tiếp S_2 hoặc S_3.

b) Trích xuất kế hoạch (Plan Extraction):

• Kế hoạch song song tại S_1: Chọn đồng thời ba hành động không mutex ở A_0 là {op3, op4, op5}. Vì vậy, bước thực hiện duy nhất là Step 1: {op3, op4, op5}.

• Kế hoạch tuần tự tương ứng: Có thể sắp xếp ba hành động theo các thứ tự khác nhau, chẳng hạn [op3 → op4 → op5] hoặc [op4 → op3 → op5]. Ngoài ra, nếu xét phương án đi qua B trên đồ thị tuần tự, ta có kế hoạch [op1 → op2].

c) Tính tối ưu của kế hoạch:

Nếu xét theo độ dài của planning graph, tức số bước thời gian, phương án song song {op3, op4, op5} là tối ưu vì hoàn thành mục tiêu chỉ sau một bước (A_0). Còn nếu tính theo tổng số hành động trong kế hoạch tuần tự, chuỗi [op1 → op2] chỉ cần hai thao tác, ít hơn phương án gồm ba thao tác.

d) Chi tiết các cặp Mutex ở mức A_0 và S_1:

Các cặp mutex ở mức A_0 (5 cặp)

1. (op6, op1): thuộc loại Interference. op6 tạo ¬A, tức làm mất A vốn là tiền điều kiện của op1.
2. (op6, op3): thuộc loại Interference vì op6 tạo ¬A, phủ định tiền điều kiện A của op3.
3. (op6, op4): thuộc loại Interference vì op6 tạo ¬A, trái với tiền điều kiện A của op4.
4. (op6, op5): thuộc loại Interference vì op6 tạo ¬A, trong khi op5 yêu cầu A.
5. (op6, No-op(A)): có cả Interference và Inconsistent Effects, bởi op6 tạo ¬A còn No-op(A) duy trì A.

Do Mutex không phụ thuộc vào thứ tự của hai hành động, cặp (op6, op5) và (op5, op6) chỉ được tính là một cặp duy nhất.

Các cặp mutex ở mức S_1 (4 cặp)

1. (A, ¬A): hai literal phủ định trực tiếp lẫn nhau.
2. (B, ¬A): B chỉ được tạo bởi op1, còn ¬A chỉ được tạo bởi op6; hai hành động này mutex.
3. (C, ¬A): op3 có quan hệ mutex với op6.
4. (D, ¬A): op4 có quan hệ mutex với op6.

(E, ¬A) không mutex vì cả hai đều có thể được tạo bởi op6, ngoài ra E còn có thể được tạo bởi op5.

Bài 6. Câu hỏi lý thuyết mở rộng

a) Trường hợp GraphPlan hoạt động hiệu quả hơn tìm kiếm không gian trạng thái:
GraphPlan thường phát huy hiệu quả với những bài toán có nhiều hành động độc lập, có khả năng thực hiện song song và có không gian trạng thái lớn. Thông qua cơ chế lan truyền Mutex (mutex propagation), thuật toán sớm nhận ra những tổ hợp hành động không thể cùng xảy ra, từ đó hạn chế việc xem xét các nhánh không khả thi thay vì phải duyệt toàn bộ không gian trạng thái.

b) Vai trò của hành động No-op và hậu quả khi bỏ No-op:

No-op dùng để biểu diễn việc một literal tiếp tục được duy trì từ mức S_i sang S_{i+1} khi không có hành động nào làm thay đổi nó. Nếu loại bỏ No-op, những mệnh đề không bị tác động cũng sẽ không được chuyển sang mức kế tiếp. Điều này làm đứt quãng sự liên tục của trạng thái trong planning graph và có thể khiến việc kiểm tra tiền điều kiện ở các bước sau không còn chính xác.

c) Vai trò của Mutex giữa các literal mục tiêu:

Khi có ít nhất một cặp literal mục tiêu bị mutex tại mức S_i, các mục tiêu đó chưa thể cùng đạt được ở bước thời gian tương ứng. Vì vậy, GraphPlan cần tiếp tục mở rộng đồ thị sang S_{i+1}. Nếu đồ thị đã "leveled off" mà một literal mục tiêu vẫn chưa xuất hiện hoặc vẫn còn cặp mục tiêu mutex, có thể kết luận bài toán không có lời giải.

d) Quy trình tổng quát trích xuất kế hoạch (Backward Search):

1. Khởi tạo: Xét mức trạng thái S_k, tại đó tất cả literal mục tiêu đều đã xuất hiện và không có cặp nào mutex với nhau.

2. Chọn hành động: Ở mức A_{k-1}, tìm một tập hành động đôi một không mutex, sao cho các Add list của chúng bao phủ toàn bộ literal mục tiêu cần đạt tại S_k.

3. Xác định mục tiêu trung gian: Đưa các tiền điều kiện (Preconditions) của những hành động đã chọn thành tập mục tiêu mới tại mức S_{k-1}.

4. Tiếp tục tìm lùi và quay lui: Lặp lại quá trình trên cho đến khi quay về S_0. Nếu không tìm được tập hành động phù hợp do gặp mâu thuẫn hoặc bế tắc, thực hiện backtrack để thử một lựa chọn khác.

5. Lưu ý: Việc các literal mục tiêu không mutex từng đôi mới chỉ là điều kiện cần, chưa đủ để khẳng định chắc chắn tồn tại kế hoạch. Nếu quá trình trích xuất không thành công, GraphPlan sẽ mở rộng thêm một mức rồi thử lại. Những tập mục tiêu đã thất bại được lưu dưới dạng nogood/memoization nhằm tránh lặp lại cùng một quá trình tìm kiếm.